

Veri Yapıları Ve Algoritmalar

KUYRUK VERİ YAPISI

Hafta 5

28.03.2023

HEDEF

- Kuyruk veri modelinin tanımı öğrenme
- Kuyruk veri modelinde bulunması gereken temel işlevleri öğrenmek
- Dizi ve bağlantılı liste üzerinde kuyruk kullanımını öğrenme

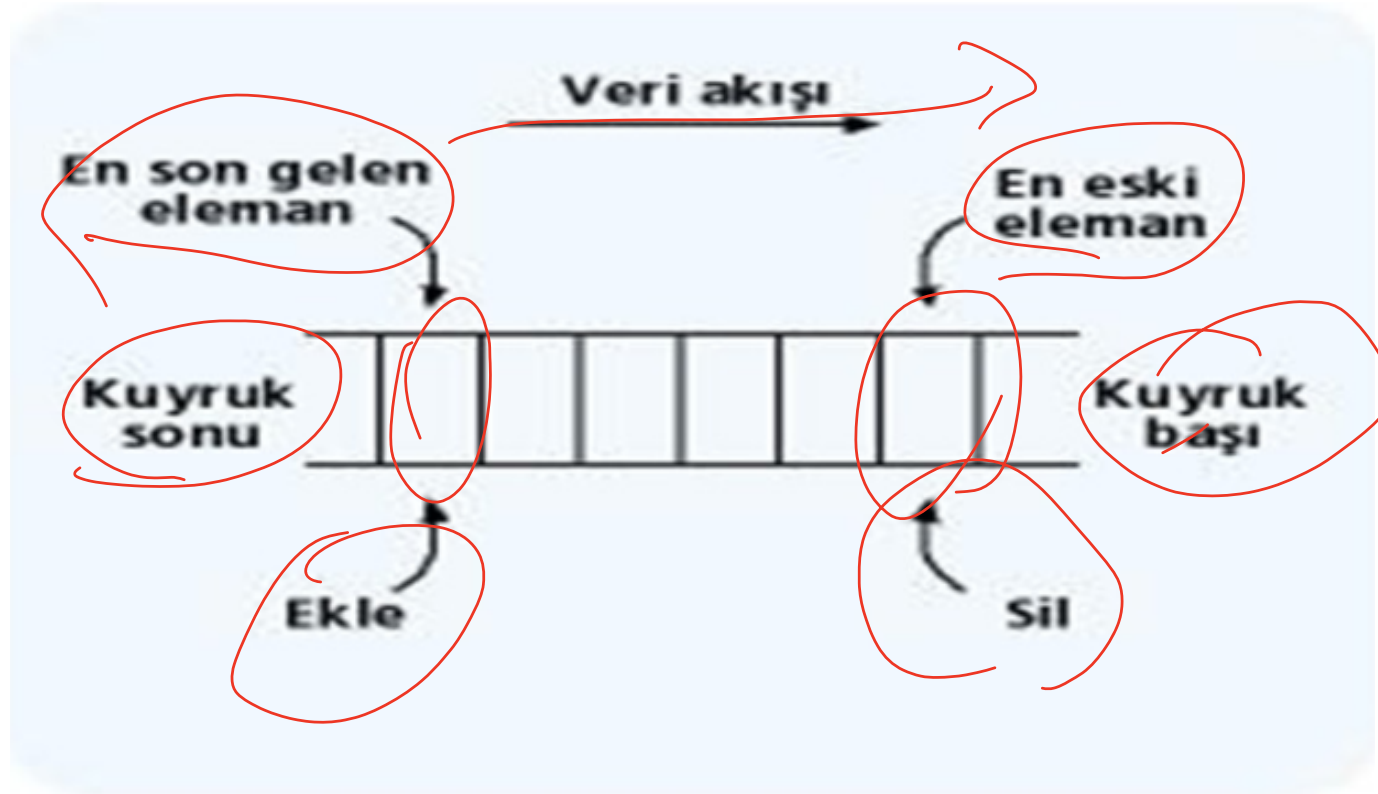
KUYRUK



GİRİŞ

- Kuyruk (queue) veri yapısı yığın (stack) yapısına benzeyen bir veri yapısıdır.
 - Kuyruk veri yapısında da iki tane işlem tanımlıdır.
 - Kuyruğa veri ekleme (enqueue) *push*
 - Kuyruktan veri çıkarma (dequeue) *pop*
 - Yığınların tersine FIFO (First In First Out) prensibine göre çalışırlar
 - Ara elemanlara erişim doğrudan yapılamaz.
 - Veri ekleme stack'teki gibi listenin sonuna eklenir.
 - Fakat veri çıkarılacağı zaman yapının son elemanı değil ilk elemanı çıkarılır.
 - Bu yüzden kuyruk veri yapısının ilk giren ilk çıkar prensibiyle çalışan bir veri yapısıdır

GİRİŞ

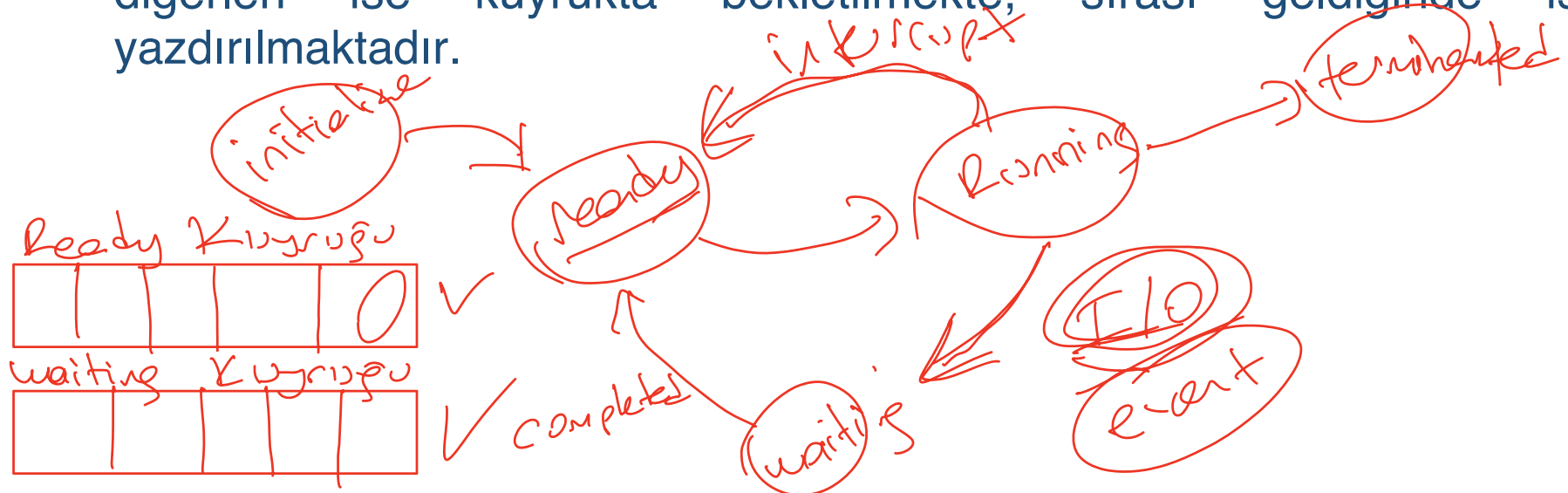


GİRİŞ

- Kuyruğun çalışma mantığını anlatan gündelik hayatta karşılaştığımız olaylara birkaç örnek verirsek;
 - Sinema gişesinden bilet almak için bekleyen insanlar,
 - ATM cihazlarından para çekmek için bekleyen insanlar, gibi, kuyruğa ilk gelen kişi ilk hizmeti alır (ilk işlem görür) ve kuyruktan çıkar

Bilgisayar Bilimlerinde Kuyruk Yapısı

- Kuyruk (queue) yapısı bilgisayar alanında;
 - Bilgisayar ağları, işletim sistemleri, istatistiksel hesaplamalar, simülasyon ve çoklu ortam uygulamalarında yaygın olarak kullanılmaktadır.
- Örneğin, yazıcılar kuyruk yapısıyla işlemleri gerçekleştirmektedir.
 - Yazdır komutu ile verilen birden fazla belgeden ilki yazdırılmakta, diğerleri ise kuyruktaki bekletilmekte, sırası geldiğinde ise yazdırılmaktadır.



Kuyruk İşlemleri

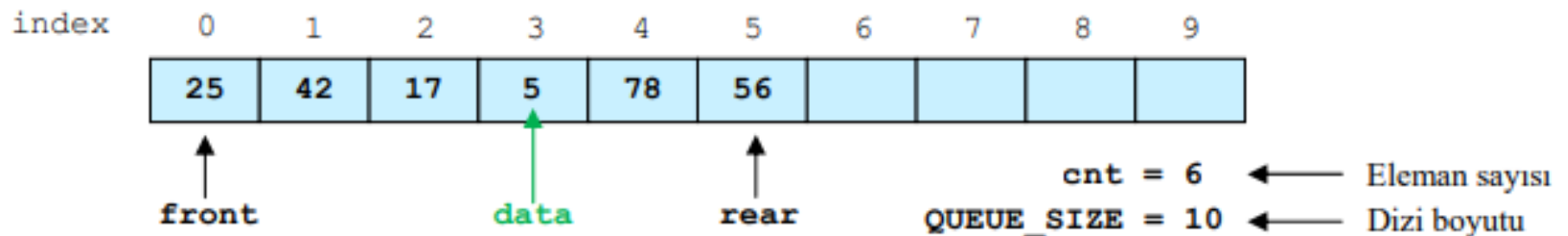
- Kuyruk (queue) yapısında da yığın yapısında olduğu gibi eleman ekleme ve eleman çıkarma olmak üzere iki işlem söz konusudur.
 - Kuyruğa eleman eklemek için kuyruğun dolu olmaması gerekir
 - ilk olarak kuyruğun dolu olup olmadığı kontrol edilir ve boş yer varsa eleman eklenir;
 - boş yer yoksa ekleme işlemi başarısız olur.
 - Kuyruğa eleman ekleme işlemi ENQUEUE olarak adlandırılır.
 - Kuyruktan eleman çıkarmak için de kuyruğun boş olmaması gerekir.
 - Bu çıkarma işlemi için de ilk olarak kuyruğun boş olup olmadığı kontrol edilir ve kuyruk boş değilse, eleman çıkarma işlemi gerçekleştirilir.
 - Eğer boşsa eleman çıkarma işlemi başarısız olur.
 - Kuyruktan eleman çıkarma işlemi DEQUEUE olarak bilinir.

Kuyruk Dizi Uygulaması

- Müşteri kuyruklarında öndeki müşteri beklediği hizmeti alıp kuyruktaki yerinden ayrılınca,
 - kuyruktakiler birer adım ön tarafa yürürler.
 - Yazılım içinde oluşturulmuş olan kuyruklarda ise bu işlem kendiliğinden ve kolay uygulanır bir nitelik değildir.

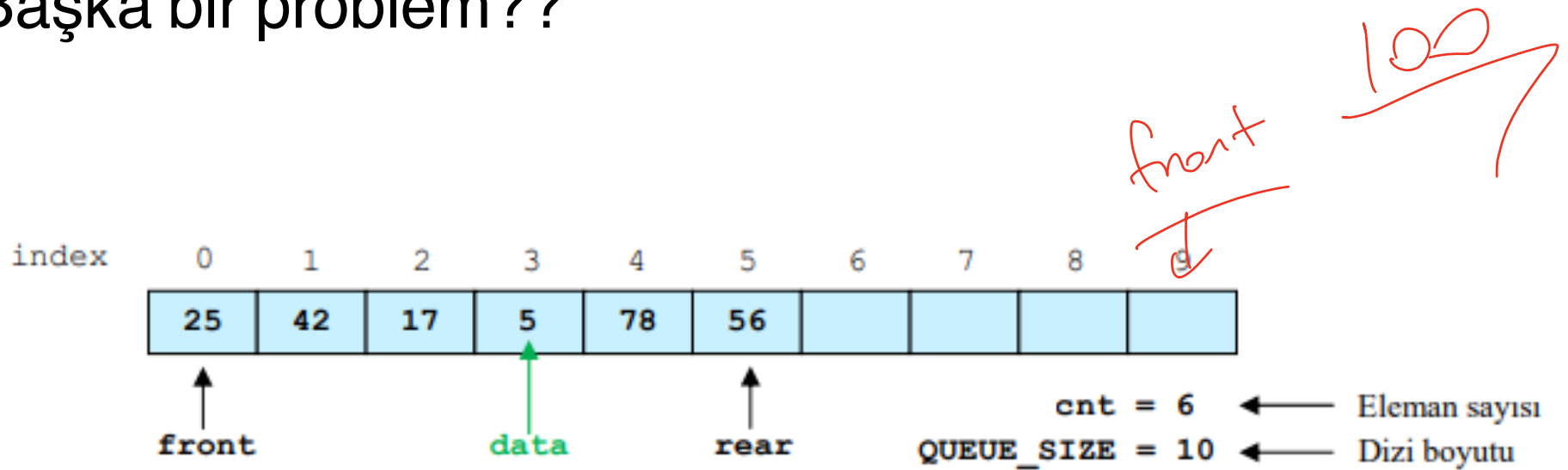
Kuyruk Dizi Uygulaması

- Basit yapılı kuyruk denen bu modelin sakıncası, kuyruktaki öğelerin sayısı kadar kaydırma işleminin kuyruktan her ayrılma olayında yapılmasıdır.
 - Ekleme ve çıkarma işlemlerinde ön indeks (front) daima sabittir.
 - Çıkarma (dequeue) işleminde, kuyruğun önündeki eleman çıkarılır ve diğer bütün elemanlar bir öne kayar,
 - ayrıca rear değişkeninin değeri de 1 azaltılır.
 - Ekleme (enqueue) işleminde ise arka indeks (rear) dizide ileri doğru hareket eder.



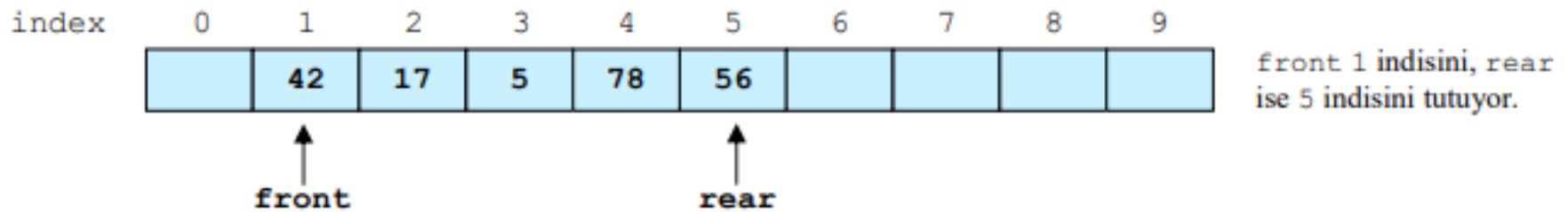
Kuyruk Dizi Uygulaması

- Sürekli kaydırma işleminin maliyeti düşünüldüğünde eleman çıkarmalarda verileri kaydırmak yerine front ön indeksi kaydırılsın şeklinde bir çözüm akla gelebilir.
- Başka bir problem??



Kuyruk Dizi Uygulaması

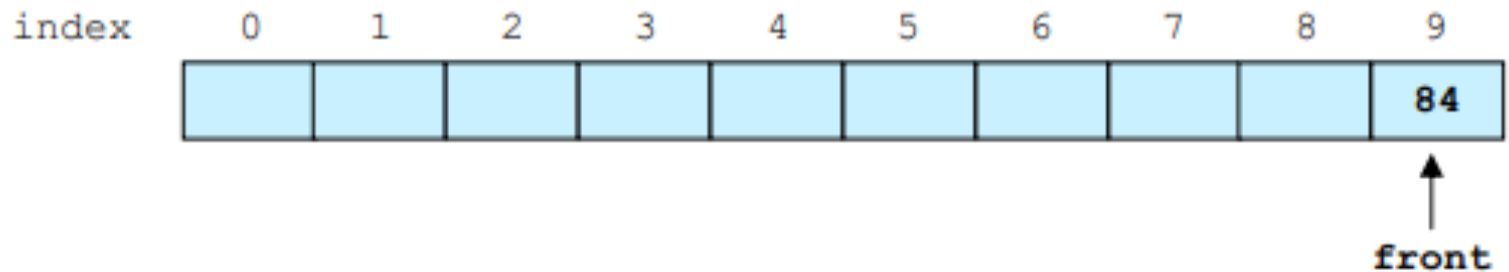
- Şekildeki gibi verileri kaydırmayıp front indeksini kaydıracağımıza göre kuyruğun solundan bir eleman çıkartıldığında orada boşluk oluşacak, front ise 1 no'lu indisi gösterecektir.



dequeue işlemi sonrası kuyruğun görünümü

Kuyruk Dizi Uygulaması

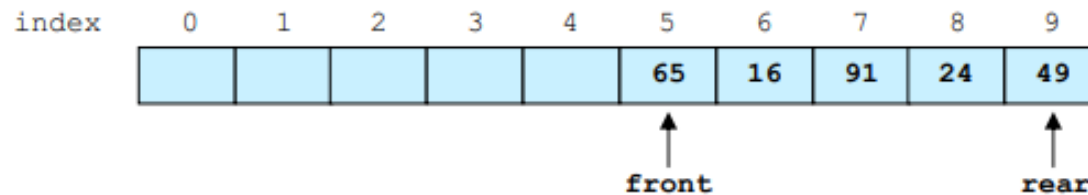
- Sürekli bir dequeue işleminde başka bir önlem alınmazsa front indeksi dizi sonuna kadar ulaşabilir. Bu durum şekilde görülmektedir.



front indeksi kuyruğun son indisini gösteriyor

Kuyruk Dizi Uygulaması

- Örneğin birkaç defa enqueue ve dequeue işlemleri sonucunda şekildeki gibi rear indeksinin kuyruğun son indisine kadar geldiğini varsayalım.
 - Eleman eklemek istediğimizde bu defa overflow hatası alınacaktır.
 - Ayrıca kuyruğun solunda bulunan boşluklara da eleman eklenemeyecektir.



Basit yapılı kuyrukta meydana gelen problem

Kuyruk Dizi Uygulaması

- Dizi ile yapılan uygulamadaki sıkıntıyı ortadan kaldırmak için, kuyruğa dairesel dizi (circular array) efekti verilebilir.
- Kuyruk işlemlerinde sürekli baştan ya da sondan çıkarma veya ekleme yapıldığına göre elemanlar arasında boşluk yok demektir.
 - Boşluklar ya baş taraftadır, ya da kuyruğun son tarafındadır.
- Kuyruklarda enqueue işleminde önce rear indeksi 1 arttırılır ve sonra eleman eklenir.
 - rear kuyruğun son indisini gösterdiğinde ekleme yapmadan önce 1 arttırıldığına göre indeks dizi boyutuna eşit olacak ve taşma hatası meydana gelecektir.
 - Bunun önüne geçmek için rear indeksi 0 olacak şekilde update edilerek elemanın kuyruğun 0 no'lu indisine eklenmesi sağlanır.

Kuyruk Dizi Uygulaması

- Kuyrukta 0 no'lu indisin boş olduğunu nereden bileceğiz?
- listeler ve yığınlarda olduğu gibi kuyruk dolu mu şeklinde bir kontrol yapılabilir.
 - Eğer dolu değilse en azından 0 no'lu indisin boş olduğu kesin olarak bilinmektedir.
 - Gerçekte dairesel bir dizi olmamasına karşın, böyle bir model şekilde görülmektedir.



Dairesel dizi (circular array) modeli.

Kuyruk - Java İle Dizi Uygulaması

- Java ile kuyruk yapısı tasarımı için aşağıdaki kodu inceleyelim. Öncelikle kuyruk sınıfımızı tasarlayalım.

```
public class kuyruk {  Sınıf başlangıcı  
    // kuyruk yapısı içinde kullanılacak değişkenler  
    private int maxBoyut; ✓  
    private long[] kuyrukDizi; ✓  
    private int front; ✓  
    private int rear; ✓  
    private int elemanSayisi; ✓
```

Kuyruk - Java İle Dizi Uygulaması

- Ardından bir kurucu metod(constructor) ile initialize işlemi gerçekleştirelim.

// constructor

// kuyruk degisken_ismi = new kuyruk (eleman_sayisi);

public kuyruk (**int** boyut){

maxBoyut = boyut;

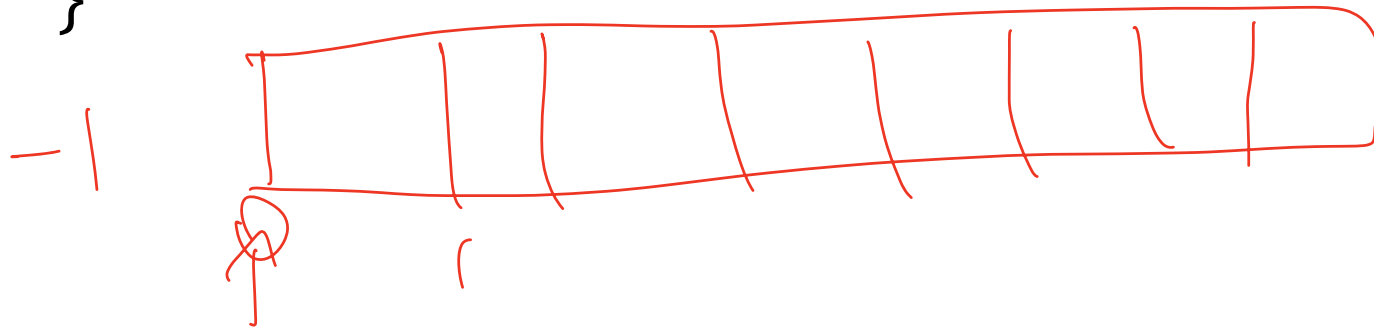
kuyrukDizi = **new long** [maxBoyut];

front = **0**;

rear = **-1**;

elemanSayisi = **0**;

}



Kuyruk - Java İle Dizi Uygulaması

- Kuyruk boş mu kontrol ediliyor.

// kuyruk boş mu?

```
public boolean bosMu() {  
    if (elemanSayisi == 0) return true;  
    else return false;  
}
```

- Kuyruk dolu mu kontrol edilmesi.

// kuyruk dolu mu?

```
public boolean doluMu() {  
    if (elemanSayisi == maxBoyut) return true;  
    else return false;  
}
```

Kuyruk - Java İle Dizi Uygulaması

- Kuyruğa eleman ekleme ve silme

// kuyruğa eleman ekleme

```
public void insert (long eklenecekEleman){
```

```
    if (rear == maxBoyut - 1) rear = -1;
```

```
    rear++;
```

```
    kuyrukDizi [rear] = eklenecekEleman;
```

```
    elemanSayisi++;
```

```
}
```

// kuyruktan eleman çekme

```
public long remove(){
```

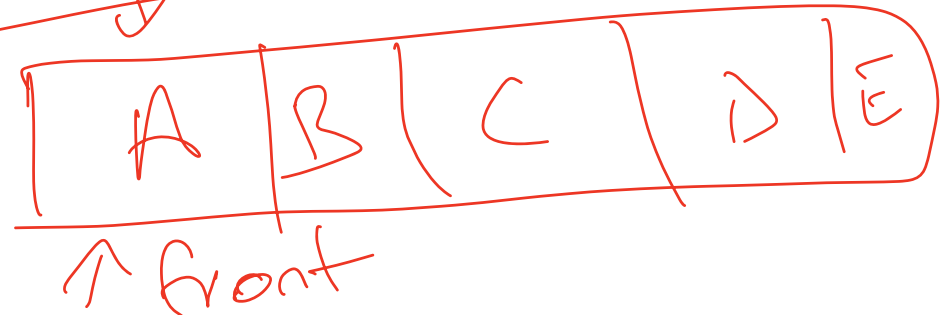
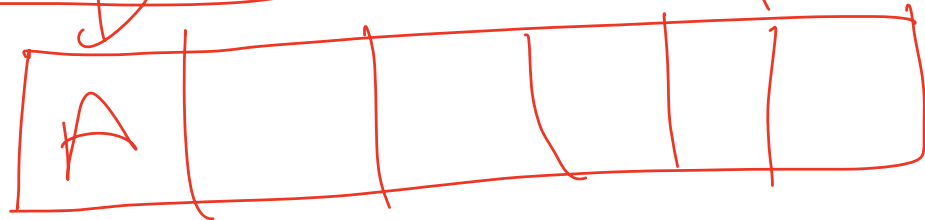
```
    long cikarilacak = kuyrukDizi [front++];
```

```
    if (front == maxBoyut) front = 0;
```

```
    elemanSayisi--;
```

```
    return cikarilacak;
```

```
}
```



Kuyruk - Java İle Dizi Uygulaması

- Kuyruğun sonundaki elemanın çekilmesi ve eleman sayısının döndürülmesi

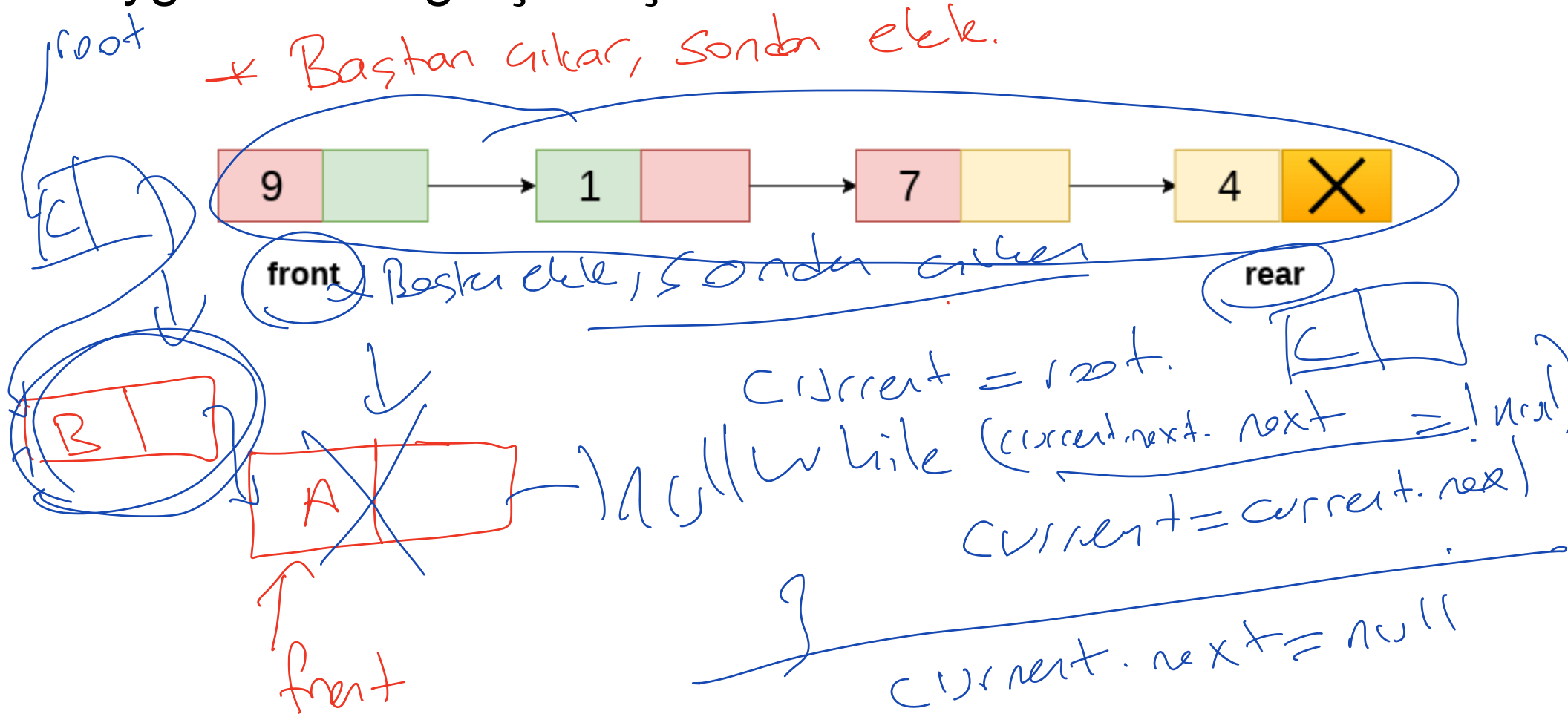
```
// kuyruğun sonundaki eleman  
public long kuyrukNerde() {  
    return kuyrukDizi[front];  
}
```

```
// kuyruğun boyutu  
public int kuyrukBoyutu() {  
    return elemanSayisi;  
}  
} Sınıf sonu
```

- Yazdığımız kodun test sınıfını yazarak deneyebilirsiniz.

Kuyruk - Bağlı Liste Uygulaması

- Şekildeki gibi kuyruk yapısını bağlı liste üzerinde uygulamasın gerçekleştirebiliriz.



Kuyruk - Bağlı Liste Uygulaması

- Kuyrukların bağı listele ile uygulamasında Aşağıdaki iki ana işlem verimli bir şekilde gerçekleştirilmelidir.
 - Kuyruk veri yapısında, ön ve arka olmak üzere iki işaretçi tutulduğunu,
 - Ön sıradaki ilk ögeyi ve arka ise son ögeyi gösterdiğini daha önce belirtmiştik.
- // Kuyruk girişini saklamak için bağlantılı bir liste (LL) düğümü

```
class QNode {  
    int key;  
    QNode next;
```

key = data
QNode next;

// Yeni bir bağlantılı liste düğümü oluşturmak için yapıcı

```
public QNode(int key) {  
    this.key = key;  
    this.next = null;  
}
```

newNode.next = null.



Kuyruk - Bağlı Liste Uygulaması

- // Bir kuyruğu temsil eden sınıf

// Kuyruk, ön bağlantılı listenin ön düğümünü ve arka bağlı listenin son düğümünü gösterir.

```
class Queue {  
    QNode front, rear;
```

```
    public Queue()
```

```
{
```

```
    this.front = this.rear = null;
```

```
}
```

Kuyruk - Bağlı Liste Uygulaması

// Kuyruğa eleman ekleme yöntemi.

```
void enqueue(int key)
```

```
{
```

```
    // Bağlı liste node oluşturma
```

```
    QNode temp = new QNode(key);
```

→



```
    // Kuyruk boş ise, yeni node hem front hem rear olacaktır.
```

```
    if (this.rear == null) {
```

```
        this.front = this.rear = temp;
```

```
        return;
```

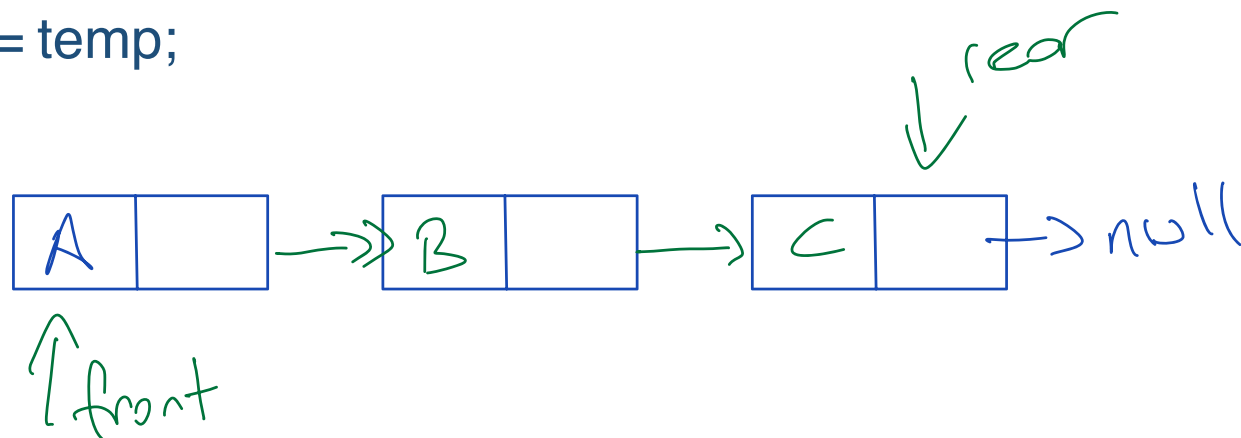
```
    }
```

```
    // Kuyruğun sonuna eleman eklenir ve rear değiştirilir.
```

```
    this.rear.next = temp;
```

```
    this.rear = temp;
```

```
}
```



Kuyruk - Bağlı Liste Uygulaması

// Kuyruktan eleman çekme

```
void dequeue()
```

```
{
```

```
    // Kuyruk boş ise, NULL döndür
```

```
    if (this.front == null)
```

```
        return;
```

```
    // Bir önceki fron değeri sakla ve bir düğüm öne git
```

```
    QNode temp = this.front;
```

```
    this.front = this.front.next;
```

Handwritten notes for the dequeue operation:

$temp = A$	$temp = B$	$temp = C$
$front = B$	$front = C$	$front = null$

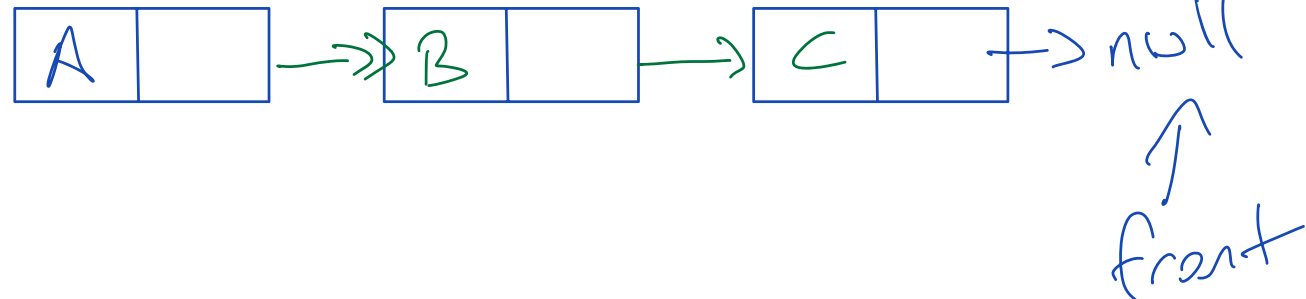
```
    // front NULL olursa, rear da NULL olarak değiştirin
```

```
    if (this.front == null)
```

```
        this.rear = null;
```

```
}
```

```
}
```



Kuyruk - Bağlı Liste Uygulaması

// Test sınıfı

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        Queue q = new Queue();
```

```
        q.enqueue(10);
```

```
        q.enqueue(20);
```

```
        q.dequeue(); ✓
```

```
        q.dequeue(); ✓
```

```
        q.enqueue(30);
```

```
        q.enqueue(40);
```

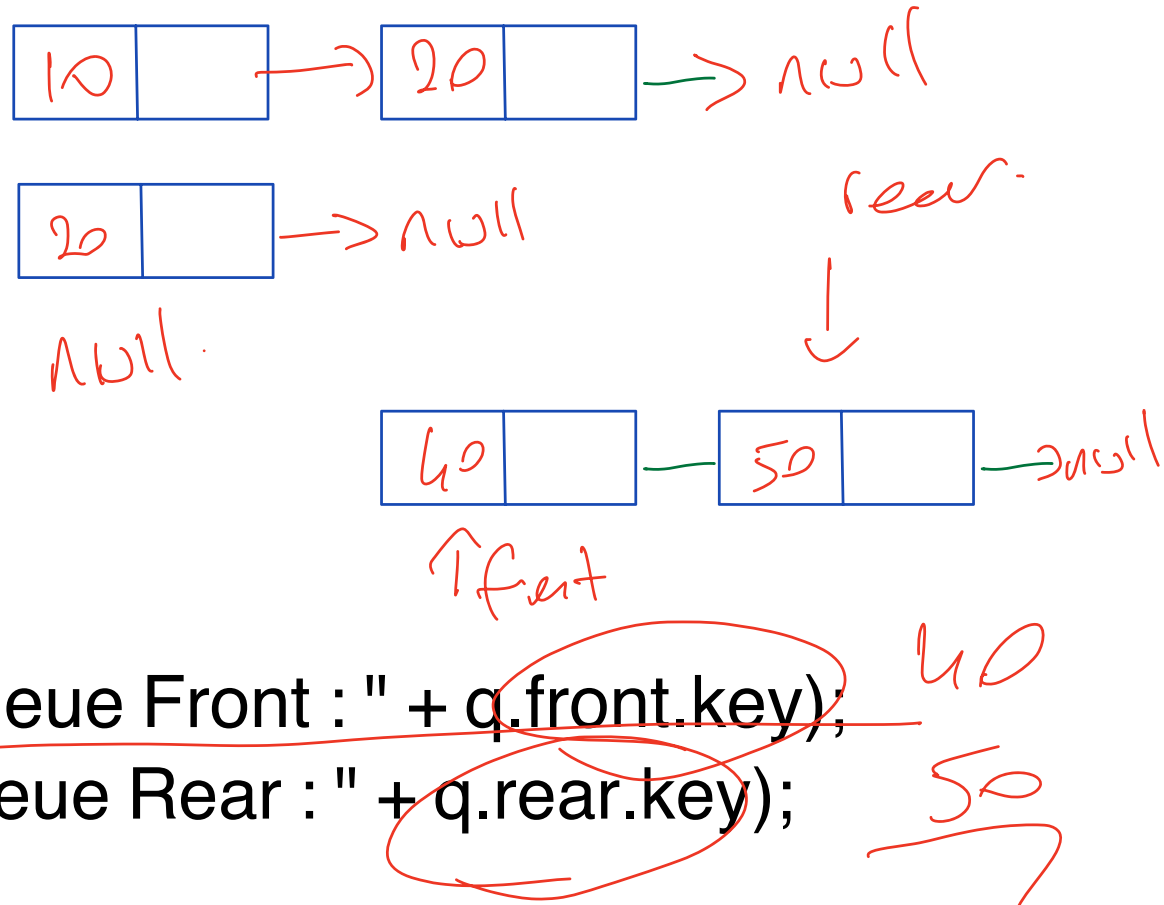
```
        q.enqueue(50);
```

```
        q.dequeue();
```

```
        System.out.println("Queue Front : " + q.front.key);
```

```
        System.out.println("Queue Rear : " + q.rear.key);
```

```
    } }
```



Kuyruk - Bağlı Liste Uygulaması

- Görüldüğü ekleme ve çekme işlemleri $O(1)$ zamanda gerçekleşmektedir. Döngü vb yoktur.

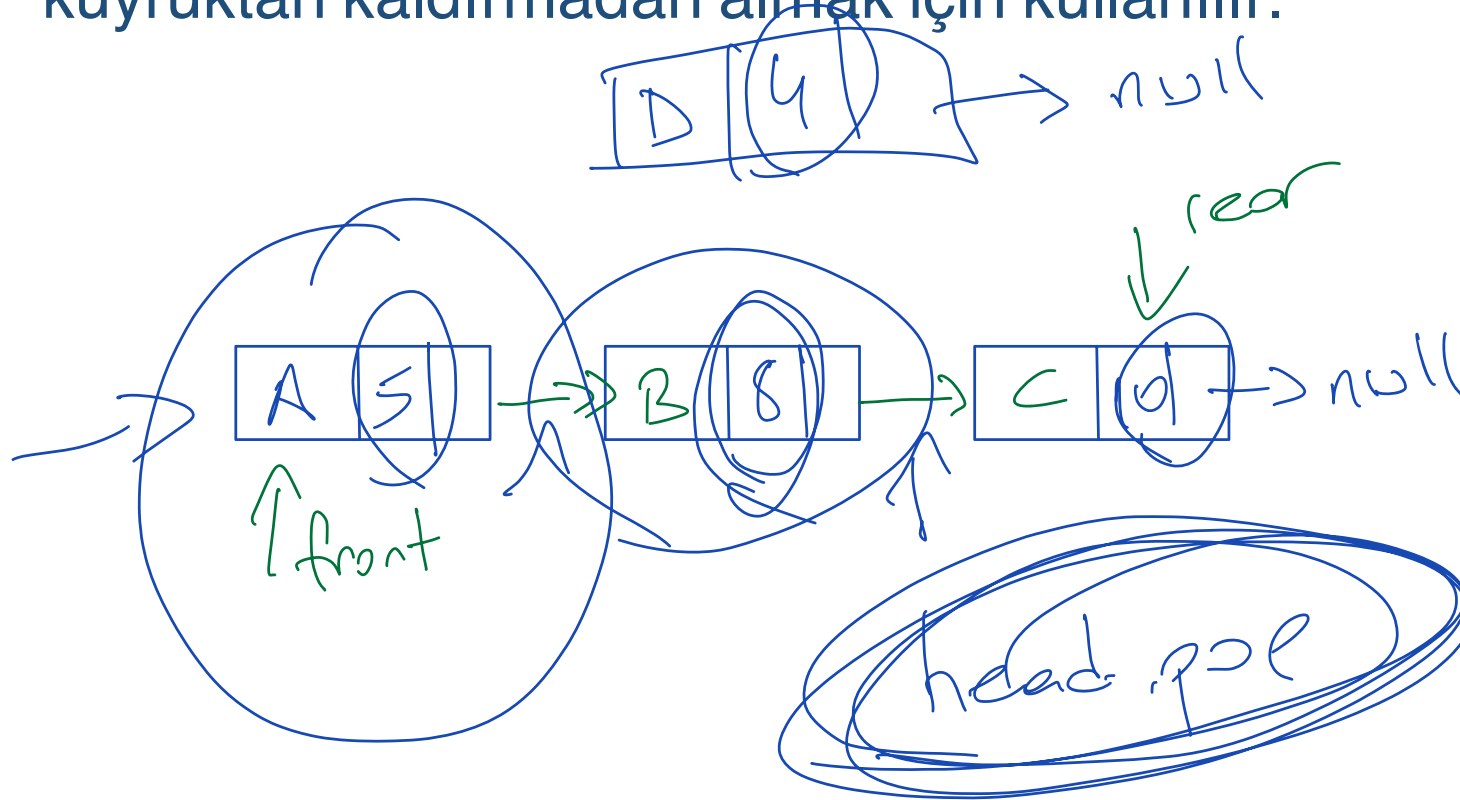
Öncelik Kuyrukları

- Öncelik Kuyruğu yapısında en önemli kısım kuyruğa eleman ekleme işlemidir.
 - Geri kalan diğer tüm işlemler normal kuyruk veri yapısı ile aynıdır.
 - Bağlı liste ile kullanımı daha doğru olduğunu öncesinde belirtmiştik.
 - Dizi ile yapmaya kalkarsanız araya her ekleme işleminde diziyi kaydırmanız gerekecektir.
- Örneğin uçakların inişi sırasında, acil inmesi gereken uçaklar bulunabilir.
 - Muayene sırasında bekleyen hastalar için farklı bir öncelik belirlenebilir.
- Bu gibi senaryolar *da öncelikli kuyruk* ile çözüm üretilir. Öncelik sırası belirlenir ve program sırasında uygulanır.

Öncelik Kuyrukları

- Öncelik Kuyruğundaki İşlemler

- push (): Bu işlev kuyruğa yeni bir veri eklemek için kullanılır.
- pop (): Bu işlev, kuyruktan en yüksek önceliğe sahip öğeyi kaldırır.
- peek () / top (): Bu işlev, kuyruktaki en yüksek öncelikli öğeyi kuyruktan kaldırmadan almak için kullanılır.



Öncelik Kuyruk Java Uygulaması

```
// Bağlı liste ile öncelik kuyruğu örnek
import java.util.* ;
class Okuyruk
{   ☐Sınıf başlangıcı
// Node
    static class Node {
        int data;

        // Düşük değerler yüksek önceliği gösterir
        int priority;

        Node next;
    }
}
```

Öncelik Kuyruk Java Uygulaması

```
static Node node = new Node();
```

```
// Yeni node oluşturulması
```

```
static Node newNode(int d, int p)
```

```
{
```

```
    Node temp = new Node();
```

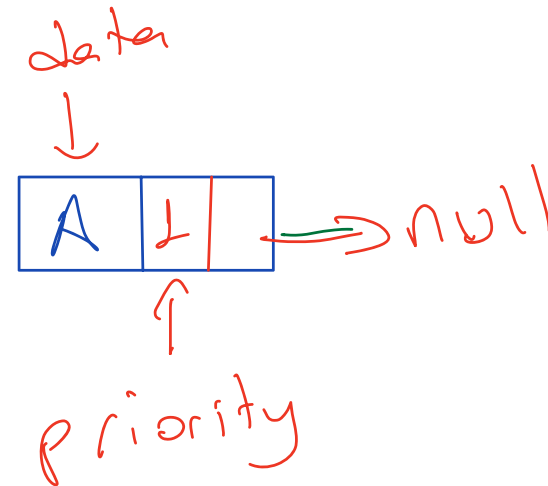
```
    temp.data = d;
```

```
    temp.priority = p;
```

```
    temp.next = null;
```

```
    return temp;
```

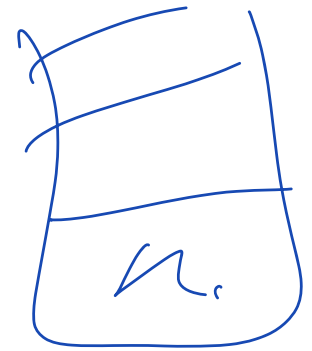
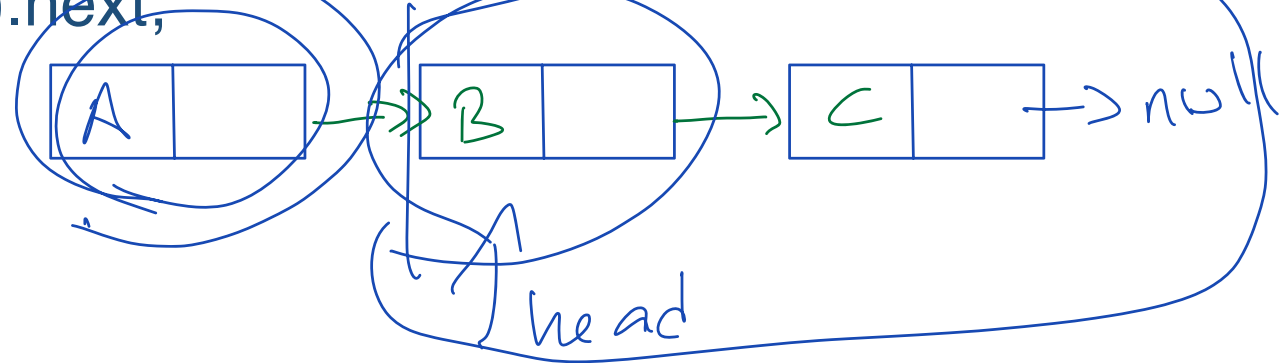
```
}
```



Öncelik Kuyruk Java Uygulaması

```
// baştaki değeri döndür
static int peek(Node head)
{
    return (head).data;
}
```

```
// En yüksek önceliğe sahip elemanı listeden kaldırır
static Node pop(Node head)
{
    Node temp = head;
    (head) = (head).next;
    return head;
}
```



Öncelik Kuyruk Java Uygulaması

// Önceliğe göre itme işlevi

```
static Node push(Node head, int d, int p){
```

```
    Node start = (head);
```

```
    // Yeni node oluşturma
```

```
    Node temp = newNode(d, p);
```

// Özel Durum: Liste başlığının yeni düğüme göre daha az önceliği vardır. Head düğümünden önce yeni düğüm eklemek ve kafa head değiştirmek için.

```
if ((head).priority > p) {
```

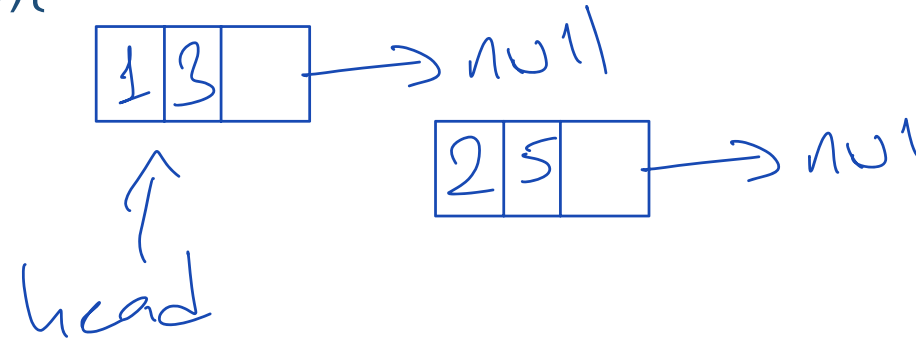
```
    // head ten önce yeni düğüm eklemek
```

```
    temp.next = head;
```

```
    (head) = temp;
```

```
}
```

```
else {
```



Öncelik Kuyruk Java Uygulaması

// Listede gezme ve yeni düğüm ekleme konumu

```
while (start.next != null &&  
      start.next.priority < p) {  
    start = start.next;  
}
```

// Ya listenin sonunda

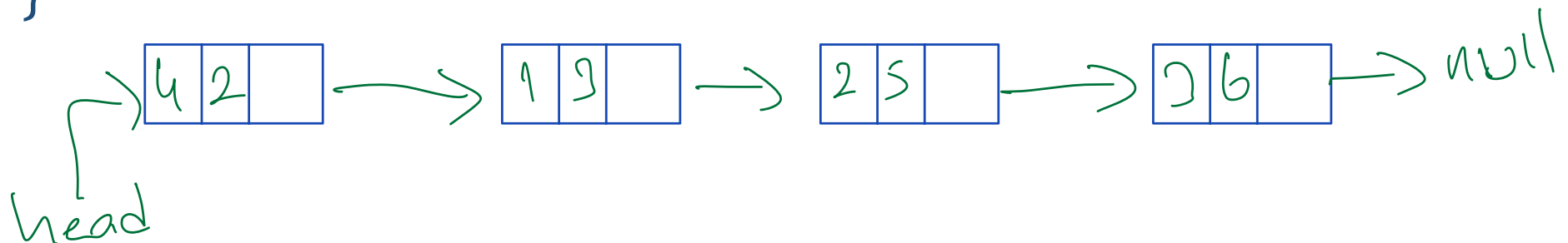
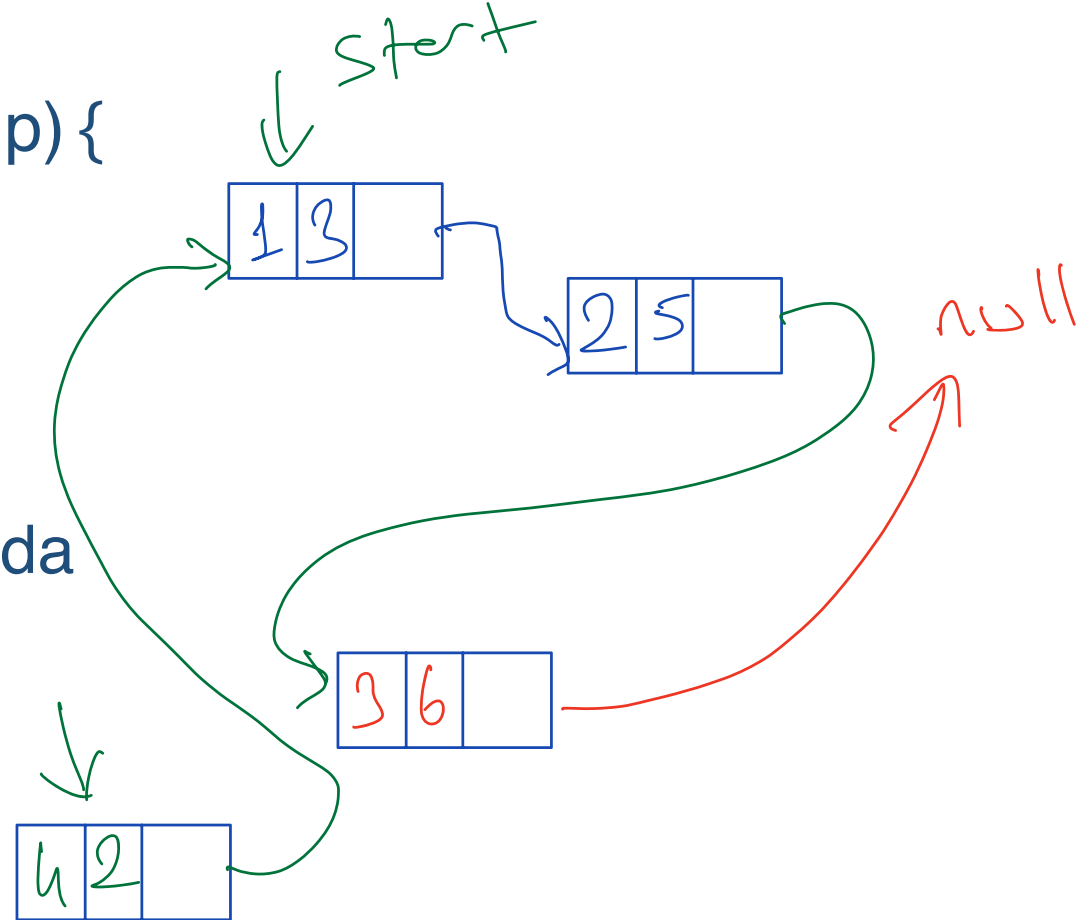
// veya gereken konumda

```
temp.next = start.next;
```

```
start.next = temp;
```

```
}  
return head;
```

```
}
```



Öncelik Kuyruk Java Uygulaması

```
// liste boş mu kontrol etme  
static int isEmpty(Node head)  
{  
    return ((head) == null)?1:0;  
}
```

Öncelik Kuyruk Java Uygulaması

// Test kodu

```
public static void main(String args[])
{
```

```
    // Öncelik kuyruğu oluşturma
```

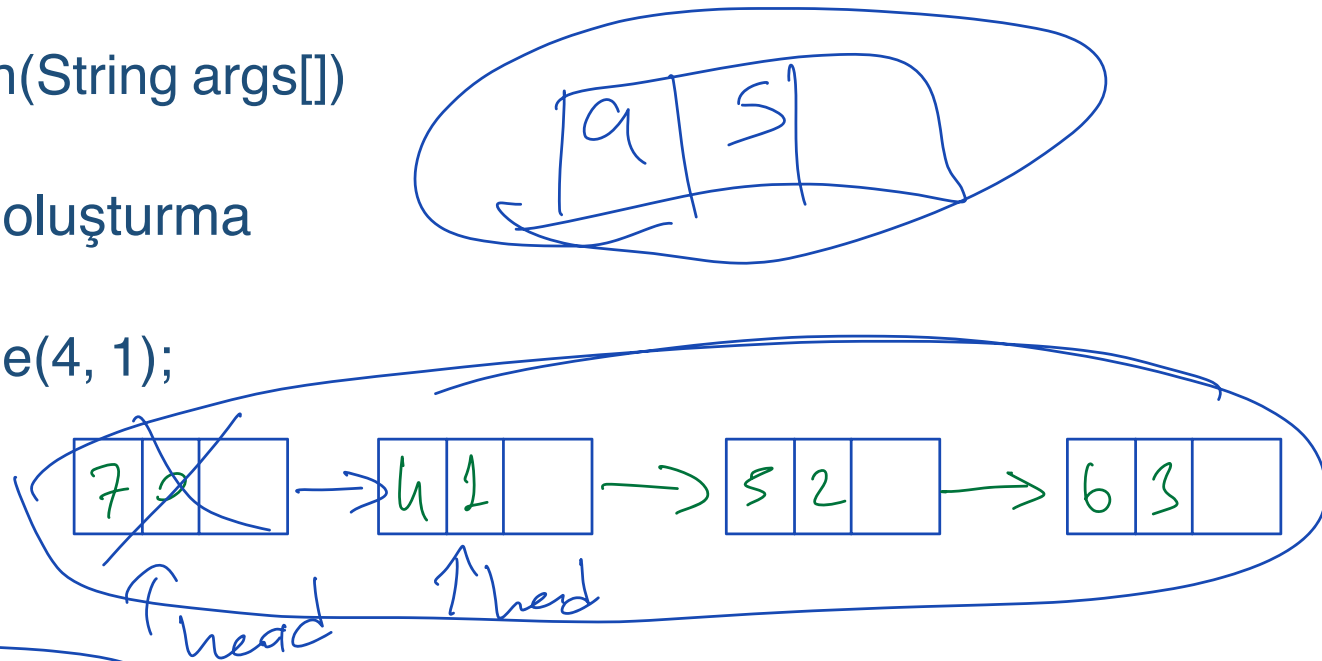
```
    // 7.4.5.6
```

```
    Node pq = newNode(4, 1);
```

```
    pq = push(pq, 5, 2);
```

```
    pq = push(pq, 6, 3);
```

```
    pq = push(pq, 7, 0);
```



```
    while (isEmpty(pq)==0) {
```

```
        System.out.printf("%d ", peek(pq));
```

```
        pq=pop(pq);
```

```
    }
```

```
}
```

```
} ☐ Sınıf bitışı
```

7, 4, 5, 6

Öncelik Kuyruk Java Uygulaması

- Asimptotik olarak bağlı liste ile öncelik kuyruğu uygulamasında peek() ve pop() işleminde $O(1)$ zaman gereklidir. Push() işleminde ise $O(n)$ zaman gereklidir.

$O(1)$

Avantajları

- Geliş sırasına göre hizmet verilmesi gereken senaryolarda avantajlıdır.
- Üretici-tüketici problemlerinde fayda sağlar.
- Hastanelerde, uçakların inişinde, araç geçişlerinde öncelikli kuyruk kullanılabilir.

Dezavantajları

- Üzerinde arama yapmak zahmetlidir. En baştan başlanıp ilerlemek gerekir.
- Kuyruğun aralarına eleman eklemek karmaşıktır.

Kullanım Alanları

- İşletim sistemlerinde çalışma önceliği kuyruk ile yapılır.
- Ağ yazıcılarında, belgeler öncelikli kuyruk ile çalışır.
- Asansör yazılımı kuyruk ile yapılabilir.

İşlem	Basit Kuyruk	Döngüsel Kuyruk	Öncelik Kuyruğu
Insert	$O(1)$	$O(1)$	$O(n)$
Remove	$O(1)$	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$	$O(1)$
IsEmpty	$O(1)$	$O(1)$	$O(1)$